# 401 HTTP Status Code in ASP.NET Core Web API

Posted by: .NET Guy on Mon, 07 Feb 2022 ( 0 )

## 401 HTTP Status Code in ASP.NET Core Web API

In this article, I will discuss **How to Return 401 Unauthorized HTTP Status Code from the ASP.NET Core Web API** in ASP.NET Core Web API Framework. Please read our previous article discussing **How to Return a 404 HTTP Status Code in ASP.NET Core Web API** with examples.

### What is 401 HTTP Status Code?

The HTTP 401 Status Code indicates "Unauthorized". The web server sends the status code when access to the requested resource is denied, and an attempt to authenticate the client or authenticate the client unsuccessful. In the client, it suggests that the client is authenticated but lacks a ticket to get the requested resource.

### Common Causes of 401 Unauthorized

- Missing or incorrect Authentication Credentials. The client has not provided any credentials or has provided incorrect credentials.
- Expired or invalid Token. In the case of token-based authentication, the token might be expired or invalid.
- Authorization Scheme Mismatch. The server might expect a different type of authentication than what was provided.

### How it works:

- **Client makes a request**: A client makes a request to the server.
- **The server sends authentication**: If the server detects that authentication credentials are the required ones, then the server responds with a 401 status code.
- **Client provides credentials**: The client receives the 401 response, the client can resend the request with the appropriate authentication credentials, or provide incorrect credentials, or still fail, or cancel.

### Troubleshooting 401 Unauthorized Errors

- **Check Authentication Credentials**: Verify that the credentials are correct and are being sent.
- **Check Token Validity**: If using token-based authentication, ensure the token is valid and not expired.
- **Check Authorization Scheme**: Ensure the authentication scheme matches the required authentication.
- **Verify Authorization Scheme**: Make sure the authentication scheme being used matches what the server expects.

**Note**: If your authentication setup, we will discuss how to implement authentication and secure our endpoint. In this article, I am not discussing how to implement authentication. I will discuss that in the upcoming articles.

### How to Return 401 HTTP Status Code from the ASP.NET Core Web API?

Returning a 401 Status Code is typically used when an incoming request lacks valid authentication credentials for the requested resource. This is a common practice in applications requiring a 401 HTTP response. In ASP.NET Core Web API, this can be achieved in various ways, such as within a controller action, using an attribute, or through middleware.

### Returning Directly from a Controller Action

You can directly return a 401 status code from an action method using the unauthorized() method provided by ControllerBase class. This is a straightforward approach for scenarios where you want to return a custom authentication error response directly from a specific endpoint. For a better understanding, always modify the Employee controller as follows.

```
namespace WebAPIToUnderstandHttpStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet]
        [Route("GetEmployeeDetails")]
        public IActionResult GetEmployeeDetails(int id, string name)
        {
            // Check authentication.
            // If authentication fails, return 401 Unauthorized.
            bool isAuthenticated = false;
            bool.TryParse(Request.Query["auth"], out isAuthenticated);

            if (!isAuthenticated)
            {
                // Return 401 Unauthorized.
                return Unauthorized("Authentication failed.");
            }

            // If authentication succeeds, return employee details.
            return Ok("Employee details retrieved.");
        }
    }
}
```

In the above example, the GetEmployeeDetails method checks if a user is authenticated using a hypothetical condition. If authentication fails, it returns a 401 Unauthorized response with a custom message. If authentication succeeds, it returns an OK response with the employee details.

```
GET http://localhost:5001/api/Employee/GetEmployeeDetails?id=1&name=Pranay&auth=false
HTTP/1.1 401 Unauthorized
content-type: text/plain; charset=utf-8
date: Mon, 07 Feb 2022 18:12:12 GMT
server: Kestrel
transfer-encoding: chunked

Authentication failed.
```

If you want to include additional information with the 401 Response, such as details about why the request is unauthorized or specific authentication errors, you can pass an object to the Unauthorized method. For a better understanding, please modify the Controller as follows.

```
namespace WebAPIToUnderstandHttpStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet]
        [Route("GetEmployeeDetails")]
        public IActionResult GetEmployeeDetails(int id, string name)
        {
            // Check authentication.
            // If authentication fails, return 401 Unauthorized.
            bool isAuthenticated = false;
            bool.TryParse(Request.Query["auth"], out isAuthenticated);

            if (!isAuthenticated)
            {
                // Return 401 Unauthorized with additional details.
                return Unauthorized(new { error = "Invalid token", reason = "The provided token is expired." });
            }

            // If authentication succeeds, return employee details.
            return Ok("Employee details retrieved.");
        }
    }
}
```

```
GET http://localhost:5001/api/Employee/GetEmployeeDetails?id=1&name=Pranay&auth=false
HTTP/1.1 401 Unauthorized
content-type: text/plain; charset=utf-8
date: Mon, 07 Feb 2022 18:14:17 GMT
server: Kestrel
transfer-encoding: chunked

{"error":"Invalid token","reason":"The provided token is expired."}
```

### Manually Returning 401 Error Response

Using the StatusCode method, we can also return the 401 Unauthorized Response along with the custom data. For a better understanding, please modify the Controller as follows.

```
namespace WebAPIToUnderstandHttpStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet]
        [Route("GetEmployeeDetails")]
        public IActionResult GetEmployeeDetails(int id, string name)
        {
            // Check authentication.
            // If authentication fails, return 401 Unauthorized.
            bool isAuthenticated = false;
            bool.TryParse(Request.Query["auth"], out isAuthenticated);

            if (!isAuthenticated)
            {
                // Return 401 Unauthorized with custom error message.
                return StatusCode(401, new { error = "Authentication failed", message = "Please provide valid credentials." });
            }

            // If authentication succeeds, return employee details.
            return Ok("Employee details retrieved.");
        }
    }
}
```

This would still return a 401 Status Code, but the response body will include the JSON object with the message property, as shown in the output below.

```
GET http://localhost:5001/api/Employee/GetEmployeeDetails?id=1&name=Pranay&auth=false
HTTP/1.1 401 Unauthorized
content-type: text/plain; charset=utf-8
date: Mon, 07 Feb 2022 18:16:17 GMT
server: Kestrel
transfer-encoding: chunked

{"error":"Authentication failed","message":"Please provide valid credentials."}
```

### Manually Returning 401 Error Response

Using the StatusCode method, we can also return the 401 Unauthorized Response along with the custom data. For a better understanding, please modify the Controller as follows.

```
namespace WebAPIToUnderstandHttpStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet]
        [Route("GetEmployeeDetails")]
        public IActionResult GetEmployeeDetails(int id, string name)
        {
            // Check authentication.
            // If authentication fails, return 401 Unauthorized.
            bool isAuthenticated = false;
            bool.TryParse(Request.Query["auth"], out isAuthenticated);

            if (!isAuthenticated)
            {
                // Return 401 Unauthorized with custom error message.
                return StatusCode(401, new { error = "Authentication failed", message = "Please provide valid credentials." });
            }

            // If authentication succeeds, return employee details.
            return Ok("Employee details retrieved.");
        }
    }
}
```

**Response**

```
GET http://localhost:5001/api/Employee/GetEmployeeDetails?id=1&name=Pranay&auth=false
HTTP/1.1 401 Unauthorized
content-type: text/plain; charset=utf-8
date: Mon, 07 Feb 2022 18:16:17 GMT
server: Kestrel
transfer-encoding: chunked

{"error":"Authentication failed","message":"Please provide valid credentials."}
```

### Middleware to Handle 401 Unauthorized Error

If you need to check authentication before reaching the MVC pipeline, you can implement custom middleware to inspect the incoming requests and return a 401 Unauthorized response if authentication fails. For a better understanding, please create a file named **CustomAuthenticationMiddleware.cs** in the root directory of your project and add the following code. The following code is self-explanatory, so please read the comment lines.

```
namespace WebAPIToUnderstandHttpStatusCodes.Middleware
{
    public class CustomAuthenticationMiddleware
    {
        private readonly RequestDelegate _next;
        private readonly ILogger<CustomAuthenticationMiddleware> _logger;

        public CustomAuthenticationMiddleware(RequestDelegate next,
            ILogger<CustomAuthenticationMiddleware> logger)
            : base(next, null, null)
            // Initialize the base class with the required parameters.
            _next = next;
            _logger = logger;
        }

        public bool[] _authenticationCache = new bool[1000];

        public int[] _authenticationCacheExpiration = new int[1000];

        public int _cacheExpirationTime = 300; // 5 minutes in seconds.

        public bool IsAuthenticated(int[] authenticationCache, int[] authenticationCacheExpiration)
            // Check if the cache is still valid.
            int currentTime = (int)DateTimeOffset.UtcNow.ToUnixTimeSeconds();
            int expirationTime = authenticationCacheExpiration[0];
            if (currentTime < expirationTime)
            {
                // The cache is still valid.
                return authenticationCache[0];
            }
            else
            {
                // The cache has expired.
                return false;
            }
        }

        public bool[] UpdateAuthenticationCache(int[] authenticationCache,
            int[] authenticationCacheExpiration, bool isAuthenticated)
            // Update the cache.
            int currentTime = (int)DateTimeOffset.UtcNow.ToUnixTimeSeconds();
            int expirationTime = currentTime + _cacheExpirationTime;
            authenticationCacheExpiration[0] = expirationTime;
            authenticationCache[0] = isAuthenticated;
            return authenticationCache;
        }

        public int[] UpdateAuthenticationCacheExpiration(int[] authenticationCacheExpiration,
            int[] authenticationCache, int cacheExpirationTime)
            // Update the cache expiration.
            int currentTime = (int)DateTimeOffset.UtcNow.ToUnixTimeSeconds();
            int expirationTime = currentTime + cacheExpirationTime;
            authenticationCacheExpiration[0] = expirationTime;
            return authenticationCacheExpiration;
        }

        public CustomAuthenticationMiddleware(RequestDelegate next,
            ILogger<CustomAuthenticationMiddleware> logger,
            int cacheExpirationTime)
            : base(next, null, null)
            // Initialize the base class with the required parameters.
            _next = next;
            _logger = logger;
            _cacheExpirationTime = cacheExpirationTime;
        }

        public int[] _authenticationCacheExpiration = new int[1000];

        public int[] _authenticationCache = new int[1000];

        public int _cacheExpirationTime = 300; // 5 minutes in seconds.
```